

Analysis of Algorithms Laboratory

Lab 07: Gaussian Elimination, LU Decomposition, and Heap Construction

Aispur P. Santiago

School of Mathematical and Computational Sciences

Yachay Tech University

San Miguel de Urucuquí - Imbabura, Ecuador

April 6, 2026

1 Introduction

This laboratory assignment consists of four main components:

1. Implement the forward elimination algorithm for Gaussian elimination.
2. Design and analyze the backwards substitution algorithm, demonstrating that its running time is $\Theta(n^2)$.
3. Implement an algorithm to transform a random array into a heap.
4. Implement an algorithm that solves a system of equations using LU decomposition, where the decomposition is based on Gaussian elimination.

2 Problem 1: Forward Elimination Algorithm

2.1 Algorithm Description

Forward elimination is the first phase of Gaussian elimination that transforms a system of linear equations into an upper triangular matrix. The transformation is accomplished by a sequence of elementary operations on the system's coefficient matrix that do not change the system's solution.

2.2 Pseudocode

The forward elimination algorithm works as follows:

```
for i ← 1 to n-1 do
  replace each of the subsequent rows (i.e., rows i+1, ..., n) by
  a difference between that row and an appropriate multiple
  of the i-th row to make the new coefficient in the i-th column
  of that row 0
```

More formally:

```
for i ← 1 to n do A[i, n+1] ← b[i]          // Augments the matrix
for i ← 1 to n-1 do
  for j ← i+1 to n do
    for k ← i to n+1 do
      A[j, k] ← A[j, k] - A[i, k] * A[j, i] / A[i, i]
```

3 Problem 2: Backwards Substitution Algorithm

3.1 Algorithm Description

After forward elimination produces an upper triangular matrix, backwards substitution solves for the unknowns by starting from the last equation and working backwards.

3.2 Pseudocode

```
for j ← n downto 1 do
    t ← 0
    for k ← j + 1 to n do
        t ← t + A[j, k] * x[k]
    x[j] ← (A[j, n+1] - t) / A[j, j]
```

3.3 Time Complexity Analysis

The backwards substitution algorithm consists of nested loops:

- Outer loop: iterates from $j = n$ down to $j = 1$, which is n iterations.
- Inner loop: for each j , the loop from $k = j + 1$ to n executes $(n - j)$ times.
- Total operations: $\sum_{j=1}^n (n - j + 1) = \sum_{i=1}^n i = \frac{n(n+1)}{2}$

This sum simplifies to $\Theta(n^2)$, as the quadratic term dominates.

4 Problem 3: Heap Construction from Random Array

4.1 Algorithm Description

A heap is constructed from a random array by applying the heapify operation. We use the bottom-up approach, starting from the last non-leaf node and moving towards the root.

4.2 Pseudocode and Efficiency

The heap construction algorithm:

```
for i ← floor(n/2) downto 1 do
    heapify(A, i, n)
```

Where heapify maintains the heap property:

```
procedure heapify(A, i, n)
    largest ← i
    left ← 2*i
    right ← 2*i + 1

    if left ≤ n and A[left] > A[largest] then
        largest ← left
    if right ≤ n and A[right] > A[largest] then
        largest ← right
```

```

if largest i then
    swap(A[i], A[largest])
    heapify(A, largest, n)

```

The time complexity is $O(n)$ because most nodes are near the leaves with small heights.

5 Problem 4: System Solving with LU Decomposition

5.1 Algorithm Description

LU decomposition factors a matrix A into two matrices: L (lower triangular) and U (upper triangular), such that $A = LU$. To solve $Ax = b$, we:

1. Decompose A into L and U using Gaussian elimination
2. Solve $Ly = b$ using forward substitution
3. Solve $Ux = y$ using backward substitution

5.2 Pseudocode

LU Decomposition (Forward Elimination):

```

for i ← 1 to n do A[i, n+1] ← b[i]          // Augments the matrix
for i ← 1 to n-1 do
    for j ← i+1 to n do
        for k ← i to n+1 do
            A[j, k] ← A[j, k] - A[i, k] * A[j, i] / A[i, i]

```

The multipliers $m_{ji} = A[j, i]/A[i, i]$ form the L matrix, while the final upper triangular form gives us U .

6 Complexity Summary

Table 1: Time Complexity of Algorithms

Algorithm	Time Complexity	Space Complexity
Forward Elimination	$O(n^3)$	$O(n^2)$
Backward Substitution	$\Theta(n^2)$	$O(n)$
Heap Construction	$O(n)$	$O(1)$
LU Decomposition	$O(n^3)$	$O(n^2)$
System Solution (LU)	$O(n^3)$	$O(n^2)$

Repository

Full source code available at: [Aispur Santiago - Analysis of Algorithms, GitHub Repository](#)